School of Computer Science
University of Nottingham

COMP2013 – Developing Maintainable Software
Autumn 2026/27

Marjahan Begum & Max L. Wilson

Information

Component weight:	100%
Expected workload:	~100 hours
Release date:	19th of Oct 2026
Submission date:	8th of Jan 2027
Marks / Feedback release date:	8th of Feb 2027
Submission method:	Moodle submission, based on git commits
Late submission policy:	UoN default 5% per working day
Feedback session date:	TBC

(note marks cannot be released during exam time)

Overview

This coursework requires you to maintain and extend a java-based game. The style of the work is designed to mirror industry practice, where you join a software engineering company, and are given things to fix in, or add to, existing software. You leave the software in a better state than you found. This is the task of software maintenance. The aim is to demonstrate skills and learning outcomes taught in lectures.

- From Task 1-6 – You are completing software maintenance as a developer
- From Task 7-9 – You will be extending the software and practise both Dev and Ops of software maintenance.

During the coursework you will make versions 2, then 3, and then 4 of the software. V1 is current version (provided), V2 is a refactored version, V3 is transferred to a new framework, and V4 is where you have extended the functionality. These represent the three primary types of software maintenance: corrective and preventative (fault repair and increase quality), adaptative (environment adaption), and perfective (functionality addition/modification).

Requirements & Specifications

You will receive 10 tasks, as git issues. You are expected to resolve these git issues roughly in order.

- For best practice, you should create sub-branch from the Dev branch for each of the tasks below, and merge them back to dev as you complete the git issue.
- You should git tag a commit that represents completing the git issue and make sure that tag is pushed to the server (refer to COMP1003 for reminders, see specified tag in relevant tasks).
- You should move each git issue, when completed, to the Closed column of the issues board. This will inform us that you think the task has been 'completed'.
- You may wish to record some video explanation of each task, or take some notes, at the time of resolving each git issue, for later use in Task 10.
- Do note delete the git issues.

The full specification of these tasks is at the end of this document.

Getting Started

Like the week 3 lab, you will find yourself in a COMP2013 gitlab group for the whole class in projects.cs.nott.ac.uk, and within that, a 'group' named after your username for you alone to work in, and within that is a project with a copy of the software that needs maintenance. You should clone and work on this repository. DO NOT fork it to a new location.

Recommendations

NOTE 1: You may not be able to begin some tasks until you have been taught the relevant content in lectures.

NOTE 2: This coursework also assumes the knowledge of git taught in COMP1003

For each assessment criteria, this module looks at **how well** you do the work. You should be making good use of gitlab features (as taught in Y1, including writing useful commit messages), using Google's java style coding conventions, creating useful content in markdown, and using Javadoc structure to comment your code.

After setting up V1, after completing V2, after completing V3, and after completing V4, you should merge dev into main (without deleting the dev branch), so that main always contains the latest working version.

You should make use of the following guides

- Writing a changelog
- Writing useful git commits
- JavaFX and IntellJ
- IntelliJ TODO and FIXME

NOTE 3: A gantt chart has been provided on moodle with a recommended timeline to help you plan how to achieve this work during the semester.

Submission Instructions

This deadline is for the final completion of all git issues, and uploading a final video to moodle. You should close most git issues earlier than this. Each git issue provides an indicative due date to stay on course.

Word and time limits specified in the tasks

In industry, staff are rewarded for providing the most salient information in the most concise and clear way, clearly justified. The provided word limits (and video length) are there to help you learn to communicate important information concisely to us. Word limits are considered as part of doing each task well.

Assessment Criteria

Criteria	Weight	Tasks	What counts as meeting this criteria "well"
Use of git features	5%	All	The repository is well organised and clean. Professional branching strategies are consistently applied. The project makes exemplary use of issues (including adding useful comments to issues being resolved), milestone and labels. Git commits are done regularly and in structure that adheres to a professional approach.
Quality of code review	5%	2	Clear and concise explanation in the markdown documentation that aligns with the diagrams, and useful Javadoc. Diagrams are clearly labelled and contain evidence of understanding of the current structure. Good use of terminology and concepts taught in class.
Quality of redesign	5%	2, 7	Clear and concise explanation in the markdown documentation that aligns with the diagrams and git features. Diagrams are clearly labelled and there is evidence of design of new features in the UML diagrams. Good use of terminology and concepts taught in class.
Quality of refactoring	40%	3-6, 9	Actual code changes are implemented well, according to code conventions and taught principles, documented in Javadoc and comments within the code. Changes are clearly explained and usefully cross referenced with git commits in the markdown.
Quality of testing	10%	3-6, 9	Initial test plan, completed test plan, and tests are well organised and usefully cross referenced with git commits. Test plans and tests are aligned with taught content. The tests should have >80% code coverage. There is clear evidence of CI later in the project.
Quality of documentation	5%	All	All the documentation (markdown and javadocs) is clear and concise and well organised, and effectively communicate the content specified for each of the tasks.
Configuration of infrastructure	10%	8	The CI/CD is purposefully designed to be useful (and is evidently used in the coursework in the later stages of the project), and explanations of this structure in markdown show clear understanding and implementation of CI.
Reasoning for decisions	20%	10	A quality .mp4 video with clear explanations (including views of code) and justifications of all the bullet points under this task. The video must not be more than 6 minutes long. It must be accessible by the teaching team . Use of visual evidence from your project in the video to support your arguments. Good marks come from clear explanations of, justifications of, and reflections on your decisions in the project. No marks will given for just showing code and features without explanation.

Resolving git issues early – 5% Bonus

In industry, speed of work is considered as well as quality of work. The criteria above focus on quality. In addition to these criteria, students are able to earn a 5% bonus (+5 to your final mark, rather than multiplying your final mark by 1.05) by submitting V4 of the software (git-tag) before the end of term (Dec 11th, end of day). Any further commits after Dec 11th would remove this bonus. Note, however, a final mark is capped at 100%.

Getting help

1. Students should use the weekly 3hr labs to get help
2. As a secondary support slot, you should make use of Marjahan and Max's weekly office hours.
3. As a last resort, you should email ps-comp2013@nottingham.ac.uk – although support via email is slow and challenging.

Please use the module email instead of emailing/contacting Max and Marjahan individually.

Academic Integrity

This coursework follows the university policy for use of ChatGPT and other LLMs. This includes not submitting LLM generated content as your own for assessment (false authorship). However, you are allowed to use LLMs to assist your learning and understanding of concepts and learning outcomes.

For this coursework, you **must not**:

- Use e.g. claude to write code
- Use LLMs to generate any code that will earn marks
- Use LLMs to generate documentation that explains code

For this coursework, you can:

- Ask LLMs to explain code to you
- Ask LLMs to help debug a problem
- Ask LLMs to help you understand how to use tools like yaml, DevOps tools, gradle, or junit, etc.
- Ask LLMs to reduce/optimise the text you created for markdown files

If you do use LLMs for these purposes, you are required to enter a statement into AI-statement.md, and include a link to a log of the interactions (either an external valid link, or a include a PDF asset in your repository). If you do use any LLM-generated code, you are required to highlight which code was authored by an LLM in the same way as citing the source of code copied from the internet, so that it is not considered when your project is being marked.

Plagiarism Vivas

The module convenors reserve the right to require that a sample of students attend in-person to explain their code, either by random sample, or for cases of suspected plagiarism.

Full Specification of Tasks

Task 1 – Setup your repository properly

The aim of this first task is to get your git project setup properly for this coursework, with good initial documentation. Clone the repository and open it in IntelliJ (or your preferred editor) and set up a dev branch to work on. Redo the primary README (as below). Add any changes made in this setup phase to Changelog.md (you should keep the changelog updated throughout the coursework with dated updates describing changes to the repository). Commit these to the project and push to projects.cs.nott.ac.uk (herein called 'the server').

Deliverables:

- Properly setup repository for V1 and initial documentation.
- Add a screenshot of the software running to the git issue.
- Link the changelog and other markdown from the main README
- Git-tag with "v1", and push it to the server.

Guideline for maintaining the main README

- Project Title + Module + Your Name
- Brief project description
- How to install and run the project
- How to use the project (i.e. how to play the game)
- Credits [original authors, and if any third-party elements need acknowledging]
- Links to other key markdown files

Task 2 – Understanding the legacy code and plan version 2

Read and try to understand how the current software is working, and add code comments. You may wish to begin to convert the comments to javadocs. Use any UML diagram of your choice to visualize and communicate how the code currently works. Think about what code smells you can see in the software, what refactoring techniques you think might be suitable, how you would like to restructure it, so that the quality of the code is improved, and explain this redesign with another UML diagram of your choice. The aim of this text and the diagrams is to best communicate understanding of problems and proposed changes. Add this content to SoftwareDesign.md under the Version 1 and Version 2 section.

You may want to consider possible changes relating (but not limited) to:

- Eliminate code smells using the refactoring techniques. The following code smells may exist in the code base:
  - long method(s), magic numbers, large/god class, dead code, complicated switch statement to list a few
  - Think about ways of restricting the code to reduce LOC through better data structures
- Refactor the code to follow good OO, SOLID, DRY, KISS and CUPID principles where its relevant. There are issues in the code that do not adhere to one or more of these principles.
- Refactor the code to incorporate relevant design patterns. For example some possible patterns are: state pattern, singleton pattern, factory patterns and strategy patterns, etc.
- Restructure the code into meaningful packages
- Restructure the code into a MVC architecture

Deliverables:

- Markdown documentation (500 words, and 2 diagrams) with the diagrams embedded that 1) describes the structure of v1, and 2) describes the planned refactoring for v2 with respect to items mentioned in the list above.
- Updated code with comments for v1.
- Link to the Software Design markdown added to main README
- Git-tag with "task2"

Task 3 – Refactor to remove within-class code smells

In this task, you should do 1 or more of the following cycle: 1) identify a code smell that you want to fix, 2) identify and/or design a test that will show the game still works as expected after you fix it (explained in the markdown table), 3) implement the test, 4) change the code to fix the problem. You should make a commit each time you complete this 4-step process, and the commit message should summarise the changes.

Deliverables:

- Incremental additions to a test description table in the testing markdown, the changelog, and the software design markdown, to be refined later
- Incremental updates to the source code and tests
- Git-tag with "task3" on the git commit where you believe task 3 is complete

Task 4 – Refactor to remove code smells across classes using coding principles

In this task, you should do 1 or more of the above 4-step process, but this time with a focus on making improvements that follow a good principle. Testing to prove this point becomes harder and needs careful reasoning.

Deliverables:

- Incremental additions to the table in the testing markdown, the changelog, and the software design markdown, to be refined later
- Incremental updates to the source code and tests
- Git-tag with "task4" on the git commit where you believe task 4 is complete

Task 5 – Refactor at application level with MVC architecture and incorporate appropriate design patterns

In this task, you should again consider how you will demonstrate that the software still behaves the same after you refactor into the MVC design pattern. You should consider whether to implement any further design patterns as part of this final refactoring stage.

Deliverables:

- V2 of the code with comments now as Javadocs
- Markdown documentation (in SoftwareDesign.md) for how the code was refactored, with cross references to git commits (500 words)
- Markdown documentation (in SoftwareDesign.md) describing the final structure, using e.g. UML (250 words, 1 UML diagram)
- A test table in Testing.md explaining the purpose of the tests that you have built
- Git-tag with "v2"

Tasks 1-5 should be completed before moving on to Task 6

At this milestone in the maintenance of the software you should be at a stage where the software is working as it was **originally intended** to with no additional features. Task 6 is about environmental adaption.

Task 6 - Convert the application from java swing to JavaFX

The management team of the company have taken the decision that we should move the application to a better GUI package for future developments. You need to move the software from java swing to JavaFX. You may need to refactor some of the tests to work with the new framework too.

Deliverables:

- V3 of the code with comments in Javadocs format
- Markdown documentation (in SoftwareDesign.md) describing the changes necessary to move to JavaFX (250 words).
- Notes in the test table in Testing.md for how tests have evolved.
- Git-tag with "v3"

Task 6 should be completed before moving on to Task 7

At this milestone, you should be at a stage where the software is still working as it was **originally intended** to with no additional features. Task 6 onwards are for software evolution.

Task 7 – Design/plan additions to the software

Plan one or two additional features to the software that you intend to include in V4. This may involve additional classes or extending an existing classes/methods. You may wish to consider any of the following ideas, whilst remembering that the work should be able to demonstrate doing evolution "well" for the assessment criteria, whilst avoiding choosing something that takes too many hours of work:

- Add traditional icons for the players
- Showing houses and hotels on the board
- Adding indicators on the board to who owns properties
- Customise some of the gameplay settings, such as changing the initial money players get, or rules about whether money goes to the bank or to free parking, how many houses are needed for a hotel, and so on (feel free to look up ideas).
- Any additional new features from your own ideas

Deliverables:

- Documentation in SoftwareDesign.md including embedded lo-fi prototypes and/or UML, of the new features, including an estimated amount of time to achieve (500 words, plus reasonable number of diagrams).
- Additional test descriptions (in Testing.md) for the planned features
- Git-tag with "task7"

Task 8 – Configure integration (CI/CD) pipeline

CI/CD pipelines are useful for ensuring that your code is automatically built and tested at a regular intervals. This task will involve configuring a pipeline further using a .gitlab-ci.yml file.

So far, all it does is run junit tests for us. You will be able to make use of up to 4 more stages of continuous integration, including code quality checks, automatic documentation, automatic builds, and automatic deployments. More details of these scripts will be released later in the semester. You can do this task at the time that makes most sense to you, once the material has been taught.

Deliverables:

- Markdown in Testing.md explaining the configurations (500 words)
- Updated YAML and configuration files
- Git-tag with "task8"

Task 9 – Extend the software for the new features, using the pipeline

Extend the code to include additions you planned in Task 6. The objectives for maintaining code quality are the same as before. Your code should not have any code smells, should follow coding conventions, should use good coding principles (OO, SOLID, DRY, KISS and CUPID), should make use of relevant design patterns, and be well tested using test-driven development. Use appropriate design patterns during this refactoring.

Deliverables:

- V4 of the software (including updated Javadoc)
- Revised documentation in SoftwareDesign.md (500 words)
- Git-tag with "v4"

Task 10 – Video justifying your decisions in the coursework

Note: this issue does not need a branch, etc. but the git issue should still be moved to closed when completed.

Create a 6-minute video in the following sequence (with indicative lengths in seconds):

- Task 10.1 - Run the game to show that it builds, show different features you have added and the associated new code (30s)
- Task 10.2 Explain the final architecture of your software with respect to your code (60s)

- Task 10.3 use the prompts below to structure your video with reference to legacy code vs quality code you have refactored with good principles (OO, SOLID DRY, KISS and CUPID) (180s)
    - Explain why it was important with reference to code smell/software quality
    - Explain the code change with reference to refactoring techniques/principles where appropriate.
    - Why you did it that way as opposed to a different way (if relevant)
- Task 10.4 Explain what design patterns you used in your code and why (60s)
- Task 10.5 Explain what changes (at least 2) could be made next to further improve the quality of the software, given what you know about good quality software. (30s)

The format should be .mp4, and the video must include a view of your face in the recording. Be mindful of video size – it should be high enough resolution for us to read what you are showing (e.g. HD), but should not be too big to hog storage. Max's recommendation is to start a Zoom call with just yourself, screen share what you want to share, make sure camera and mic are on, and hit record (to local storage). However, many other approaches are available. You may wish to use a video editor approach if you are combining audio with multiple screen captures. Handbrake is then a good free app that is good at compressing videos while maintaining HD. You should make sure your video can be understood clearly – you may wish to embed subtitles in the video for clarity. Marks may be lost if we cannot understand what you are saying.

Deliverables:

- Submit the video to moodle.
- Close the final git issue.